

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Task No.: 3

Student Name: EMANDI HARI CHARAN

Roll No.: A24126552080

Date:

1. TITLE

Dynamic To-Do List Application Using JavaScript DOM

2. AIM

To design and develop an interactive To-Do List web application using HTML5, CSS3, and JavaScript DOM manipulation to manage dynamic task additions, state toggling (completion with strikethrough), task removal, and real-time empty state visibility handling without refreshing the webpage.

3. OBJECTIVE

The objectives of this experiment are:

- To understand and implement client-side Document Object Model (DOM) selection using `getElementById()`.
- To dynamically generate and configure new HTML elements (`document.createElement`) at runtime.
- To attach dynamic event listeners and onclick handlers to dynamically created UI components.
- To modify element class lists dynamically (`classList.toggle`) to update visual presentation.
- To safely remove nodes from the live DOM tree using `li.remove()`.
- To validate user inputs and automatically toggle conditional display elements based on list child count.

4. THEORY

The Document Object Model (DOM) is an object-oriented representation of an HTML document, structured as a logical tree of nodes. Every HTML tag, attribute, and text block is represented as a node object that JavaScript can query, inspect, modify, or delete.

Dynamic DOM manipulation in this application implements the following lifecycle:

- **Input Validation:** Inspects the value property of `taskInput`; if empty, prompts an alert to guarantee input integrity.
- **Node Creation:** Invokes `document.createElement('li')` and `document.createElement('button')` to assemble interactive task items dynamically in memory.
- **Tree Insertion:** Attaches child buttons to the `<li>` and appends the `<li>` to the parent `<ul>` container via `appendChild()`.

- State Toggling: Toggles the '.completed' CSS class (text-decoration: line-through) upon clicking the Complete button.
- Node Deletion: Calls li.remove() to purge the targeted element from memory and updates the empty-state indicator if children count hits zero.

Data Flow:

User Event (Click 'Add Task') → Input Sanitization → DOM Node Creation (li, button) → DOM Tree Mutation (appendChild) → Event Listener Binding → UI Render

5. ALGORITHM / PROCEDURE

1. Create index.html: Define input container with <input type='text' id='taskInput'> and <button id='addTask'>.
2. Add an empty-state paragraph <p id='emptyMessage'>No tasks available.</p> and an empty task list <ul id='taskList'>.
3. Define CSS rules for card layouts, buttons (.complete in blue, .delete in red), and completed state (.completed with line-through).
4. Create Script.js: Select target DOM nodes using document.getElementById().
5. Attach onclick handler to addTask: Read taskInput.value.trim(). If empty, alert 'Please enter a task' and return.
6. Create element, assign innerHTML = task. Create 'Complete' button with onclick handler toggling li.classList.toggle('completed').
7. Create 'Delete' button with onclick handler calling li.remove(). If taskList.children.length === 0, set emptyMessage.style.display = 'block'.
8. Append Complete and Delete buttons to ; append to taskList; clear input field; hide emptyMessage.
9. Test task addition, completion toggle, deletion, and empty state message toggling in browser.

6. PROGRAM

index.html

```
<!DOCTYPE html>
<html>

<head>
  <title>My To-Do List</title>

  <style>
    body {
      font-family: Arial;
      background-color: #f2f4f8;
      text-align: center;
      margin-top: 50px;
    }

    h1 {
      color: #333333;
```

```
}

.input-section {
  margin: 20px;
}

input {
  padding: 10px;
  width: 250px;
  border: 1px solid #cccccc;
  border-radius: 5px;
}

#addTask {
  padding: 10px 15px;
  background-color: #4CAF50;
  color: white;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}

#addTask:hover {
  background-color: #45a049;
}

#emptyMessage {
  color: #777777;
}

ul {
  list-style-type: none;
  padding: 0;
}

li {
  background-color: white;
  margin: 10px auto;
  padding: 10px;
  width: 400px;
  border-radius: 5px;
  box-shadow: 0 2px 5px #cccccc;
}

.complete {
  margin-left: 15px;
  padding: 5px 10px;
  background-color: #2196F3;
  color: white;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.delete {
```

```

        margin-left: 5px;
        padding: 5px 10px;
        background-color: #f44336;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
    }

    .complete:hover {
        background-color: #1976D2;
    }

    .delete:hover {
        background-color: #d32f2f;
    }

    /* Completed task */
    .completed {
        text-decoration: line-through;
        color: #888888;
    }
</style>
</head>

<body>

    <h1>My To-Do List</h1>

    <div class="input-section">
        <input type="text" id="taskInput" placeholder="Enter a task">
        <button id="addTask">Add Task</button>
    </div>

    <p id="emptyMessage">No tasks available.</p>

    <ul id="taskList"></ul>

    <script src="Script.js"></script>

</body>

</html>

```

Script.js

```

let taskInput = document.getElementById("taskInput");
let addTask = document.getElementById("addTask");
let taskList = document.getElementById("taskList");
let emptyMessage = document.getElementById("emptyMessage");

addTask.onclick = function () {

```

```

let task = taskInput.value;

if (task == "") {
  alert("Please enter a task");
  return;
}

// Create task
let li = document.createElement("li");

li.innerHTML = task;

// Create Complete button
let completeButton = document.createElement("button");

completeButton.innerHTML = "Complete";
completeButton.className = "complete";

// Complete task
completeButton.onclick = function () {
  li.classList.toggle("completed");
};

// Create Delete button
let deleteButton = document.createElement("button");

deleteButton.innerHTML = "Delete";
deleteButton.className = "delete";

// Delete task
deleteButton.onclick = function () {
  li.remove();

  if (taskList.children.length == 0) {
    emptyMessage.style.display = "block";
  }
};

li.appendChild(completeButton);
li.appendChild(deleteButton);

taskList.appendChild(li);

taskInput.value = "";

emptyMessage.style.display = "none";
};

```

7. CODE EXPLANATION

Code / Component	Explanation
------------------	-------------

<code>document.getElementById(id)</code>	Selects and returns a specific DOM element matching the specified unique ID attribute.
<code>taskInput.value</code>	Retrieves the current string data entered by the user into the text input control.
<code>document.createElement('li')</code>	Instantiates a new HTML list item element node in memory before DOM insertion.
<code>button.className = 'complete'</code>	Assigns pre-styled CSS classes to dynamically created button elements.
<code>li.appendChild(element)</code>	Appends a child node element as the last child to the parent <code></code> container.
<code>li.classList.toggle('completed')</code>	Toggles the presence of the 'completed' class on the target element dynamically.
<code>li.remove()</code>	Removes the targeted <code></code> node directly from the Document Object Model tree.
<code>taskList.children.length</code>	Returns the active count of child elements present within the task list container.
<code>emptyMessage.style.display</code>	Dynamically toggles CSS display property between 'none' and 'block' based on task count.

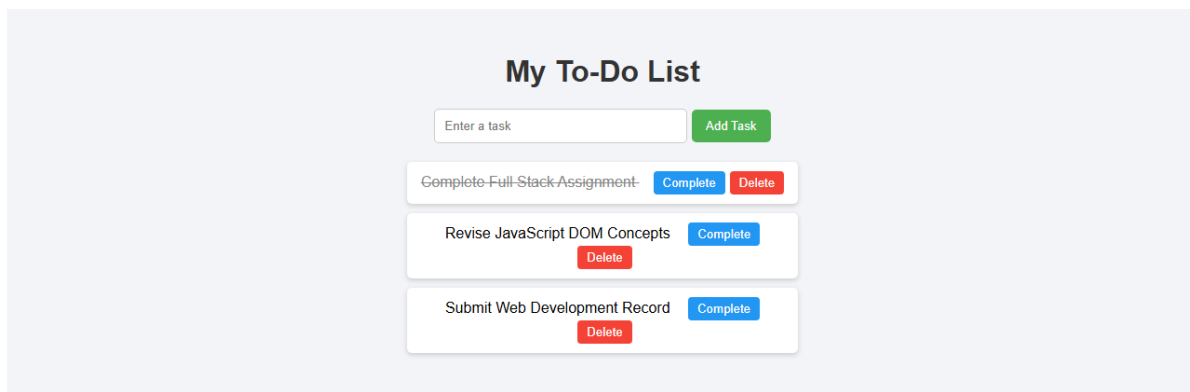
8. TEST CASES AND EXECUTION DATA

Test Case	Input Name	Test Input	Expected Result	Actual Result	Status
TC-01	Initial Empty View	Page load	'No tasks available.' text displayed; list is empty	Displayed correctly	Pass
TC-02	Empty Input Validation	Click 'Add Task' with empty input	Browser alert prompt: 'Please enter a task'	Alert displayed	Pass
TC-03	Task Addition	Input: 'Complete Full Stack Assignment'	New task <code></code> created with Complete & Delete buttons	Task added cleanly	Pass
TC-04	Complete Task	Click 'Complete'	Task text struck through with line-	Strikethrough applied	Pass

		button	through styling		
TC-05	Uncomplete Task	Click 'Complete' button again	Strikethrough removed from task text	Class toggled off	Pass
TC-06	Delete Task	Click 'Delete' button	Target task removed from DOM; empty message toggles if empty	Removed cleanly	Pass

9. OUTPUT

Output 1: Dynamic To-Do List Application Execution (Initial Empty & Interactive States)



10. RESULT

Thus, the dynamic To-Do List web application was successfully designed, developed, and verified using JavaScript DOM manipulation. Element creation, dynamic event binding, CSS class toggling, node removal, and conditional empty-state handling functioned accurately across all test cases.